Tandem Bicycle

AlgoExpert · Easy · Greedy Algorithms

The problem

- Pair red and blue riders; a tandem's speed is the max of its riders.
- Return the max (fastest=true) or min possible total speed.

Intuition

- Speed is the max of the two riders.
- For maximum total, pair the fastest with the slowest; for minimum, pair like with like.

Brute force vs optimal

- Brute force: try all n! pairings
- Fastest total: pair big with small (opposite orders)
- Slowest total: pair big with big (same order) → O(n log n)

Approach

- 1. Sort both lists
- 2. For fastest total, pair one ascending with the other descending
- 3. For slowest, pair in the same order
- 4. Sum the max of each pair

Worked example

red ascending



blue descending

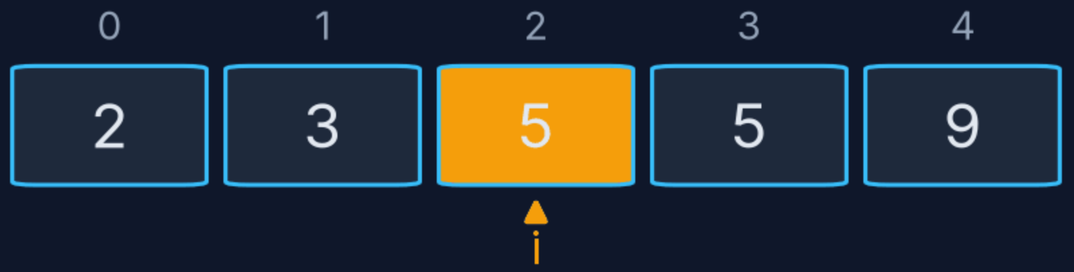



```
max(2, 7) = 7
total = 7
```

Pair 2 with 7 → +7 (total 7)

Worked example

red ascending



blue descending

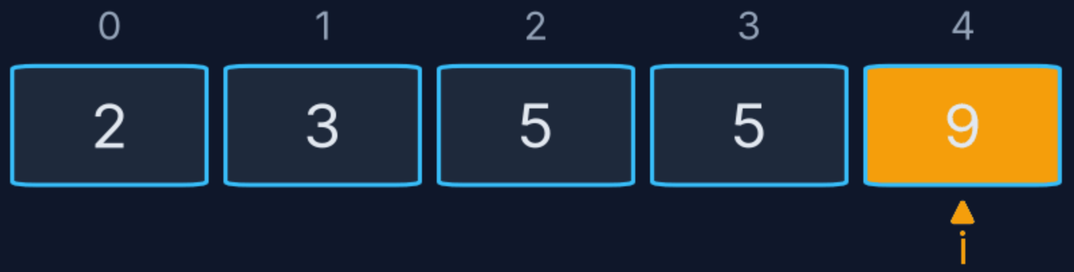



```
max(5, 3) = 5
total = 18
```

Pair 5 with 3 → +5 (total 18)

Worked example

red ascending



blue descending




```
max(9, 1) = 9
total = 32
```

Pair 9 with 1 → +9 (total 32)

Java solution

```java
public class TandemBicycle {
    public static int tandemBicycle(int[] red, int[] blue, boolean fastest) {
        Arrays.sort(red);
        Arrays.sort(blue);
        int n = red.length, total = 0;
        for (int i = 0; i < n; i++) {
            int b = fastest ? blue[n - 1 - i] : blue[i];
            total += Math.max(red[i], b);
        }
        return total;
    }
}
```

Complexity

- Time: O(n log n)
- Space: O(1)

Edge cases & pitfalls

- Fastest flag flips the sort order of one list
- Sorting mutates inputs
- Equal speeds are fine

Interview tips & follow-ups

- Justify with an exchange argument
- Follow-up: k riders per bike

Recap

- Speed is the max of the two riders.
- Complexity: O(n log n)